

Questryn — User & Customization Guide



This guide covers two things together, on purpose: how to **use** the Questryn editor to author dialogues, and how the **example Blueprint project** included with the plugin implements everything around those dialogues (conditions, text tags, rich text, sounds, widgets). They're not really separable — features like path conditions, `{Tag}` text substitution and rich text markup only work end-to-end once you've adapted the Blueprint side to your own project (your items, your data tables, your character class). Read this guide top to bottom once, then come back to individual sections as reference.

Table of Contents

1. What Qvestryn Is
 2. Getting Started: Setting Up Your Own Copy
 3. Reading the Comments: POSSIBLE CHANGE vs. CHANGE RECOMMENDATION
 4. The Editor Workflow
 5. Blueprint Architecture of the Example Project
 6. Where to Start
 7. LLM-Assisted Dialogue Authoring
 8. Sharing Your Game (Optional)
-

1. What Questryn Is

Questryn — Node-Based Dialogue System is a dialogue plugin for Unreal Engine 5. A dialogue is authored as a small **graph**: a Start Node feeds into a Dialog Node, and each Dialog Node can branch into further Dialog Nodes through labeled **paths**. Each Dialog Node holds one or more lines ("SubDialogues") — dialog title (usually the speaker), text(s), image(s), sound(s), and duration.

The graph reads top-to-bottom, like a script, instead of the left-to-right layout most Unreal graph editors use — that's a deliberate choice, so a branching conversation stays readable as a conversation.

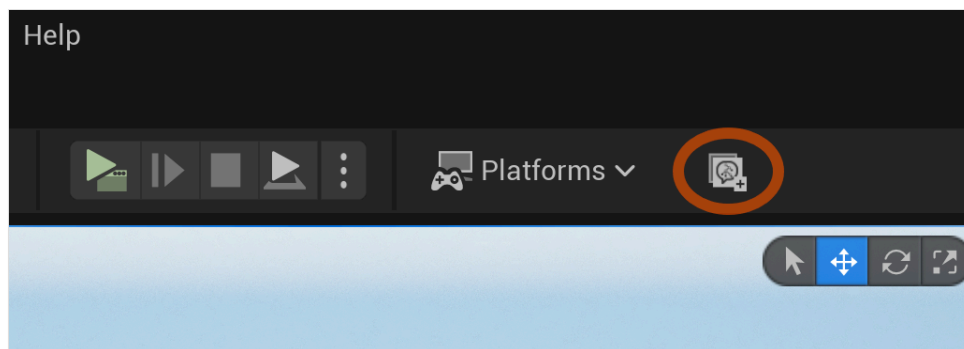
Everything you build in the graph is **data**. The plugin's C++ side owns the dialogue's actual flow — advancing through nodes, the priority/Lower Priority Behavior system, blocking actions — but hands off all *presentation* to Blueprint: which widget shows up, how text is revealed, what happens when a condition is checked, is driven by an example **Blueprint project** shipped alongside the plugin, which you're meant to adapt (Section 5).

2. Getting Started: Setting Up Your Own Copy

The plugin ships with a full set of example content — Blueprints, DataTables, Widgets, an example map — living under the `/Qwestryn/` content mount point, used to demonstrate every feature end-to-end (`PC_Qwestryn_Example` , `Character_Qwestryn_Example` , `BP_Qwestryn_Trigger_Base` , `WB_Qwestryn_Dialogue_Example` , and so on). This example content is meant to be **read, copied, and eventually replaced** — not built on top of directly.

Recommended way to start your own project:

1. **Decide what you actually need.** You rarely need all of it on day one. A common starting set is: your Player Controller's blocking-actions handling, your condition-check functions, one Trigger Blueprint (a **copy** of `BP_Qwestryn_Trigger_Base` — copying is the supported path here, not subclassing it), the main dialogue widget and its companion widgets, and the 4 Blueprint Interfaces.
2. **Copy the assets into your project.** Two ways to do this:
 - o **"Add Starter Content" button (recommended)** — a one-click shortcut for the common case. In the Level Editor toolbar, click **Add Starter Content**, pick a destination folder in your project in the picker that opens, then **Confirm**. This copies the full starter set — every Blueprint, Widget, Interface, the example Trigger Box, the example dialogues, and the supporting DataTables/Structs/Enums they depend on — into `<your folder>/QwestrynStarterContent/` (split into `Blueprints/` , `UI/` , `Data/` , `Dialogues/` and `Badge/` subfolders), with each asset already renamed with a `Starter_` prefix (e.g. `Starter_BP_Qwestryn_Trigger_Base`) and cross-references between the copies already pointing at each other — no manual "Migrate", no risk of missing a dependency. The one exception is textures and sounds: those stay referenced to the plugin's own `/Qwestryn/` example media, since they're not meant to be edited and duplicating read-only art assets wouldn't buy you anything.



It also builds you a ready-to-play `Maps/Starter_Map_Qwestryn` — a fresh level (not a copy of the plugin's own example map; bulk-copying a Level isn't reliable in UE5's asset tools) populated with the copied Trigger Box placed three times, each wired to one of the copied example dialogues, a PlayerStart, a floor, and basic lighting. Open it and press Play to test the duplicated assets immediately, fully independent of the plugin's own `/Qwestryn/` content. **Note:** creating this map replaces whatever level you currently have open — you'll get a confirmation prompt first if it has unsaved changes. If `Starter_Map_Qwestryn` already exists at the destination from a previous run, it's left alone (delete it first if you want a fresh one). If any of the copied assets already exist at the destination, you'll get a confirmation prompt before anything is overwritten. Don't need this button around? It can be hidden per-project via **Project Settings** → **Plugins** → **Qwestryn** → **"Show 'Add Starter Content' Button"**.

- o **Manual copy from inside the Editor** — use this instead if you want to hand-pick a subset rather than take the whole starter set. The example content lives at `/All/EngineData/Plugins/Qwestryn` in the Content Browser — if you'd rather click through than type a path, enable *Show Engine Content* and *Show Plugin Content* in the Content Browser's view options, then navigate **All** → **Engine** → **Plugins** → **Qwestryn - Node-Based Dialogue System Content**. From there, use copy/paste

or "Migrate..." into your own Content subfolder. Don't copy `.uasset` files outside the Editor (through the OS file explorer); Blueprint/DataTable assets are binary and editing or moving them outside the Editor risks corrupting them.

3. **Rename your copies** to match your project's naming convention (drop the `Starter_ / _Example` naming, use your own prefixes). This keeps them clearly distinct from the plugin's reference copies, and means your game no longer depends on the plugin's `/Questastryn/` example content once you've made the switch.
4. **Re-point references** in your copies: DataTable Row Struct types, the Data Table defaults used by your Dialog Nodes, which classes implement the 4 interfaces, the Widget Class / Dialog Trigger Class fields on your dialogues, etc.
5. **Only then** start working through the `CHANGE RECOMMENDATION / POSSIBLE CHANGE` comments (Section 3) with your own logic.

You don't have to do this all at once. Plenty of projects keep using `BP_Questastryn_Trigger_Base` itself for a while and only swap out the pieces they need (commonly just `CheckCondition` and `RunData`), while everything else still runs on the example implementation. One piece isn't optional, though: if you keep the example widget's rich-text rendering (the `<bigtext>`-style markup), you need to keep or adapt the Rich Text Style Table it reads from — Unreal's Rich Text Block can't resolve any markup without a valid style table assigned.

3. Reading the Comments: POSSIBLE CHANGE vs. CHANGE RECOMMENDATION

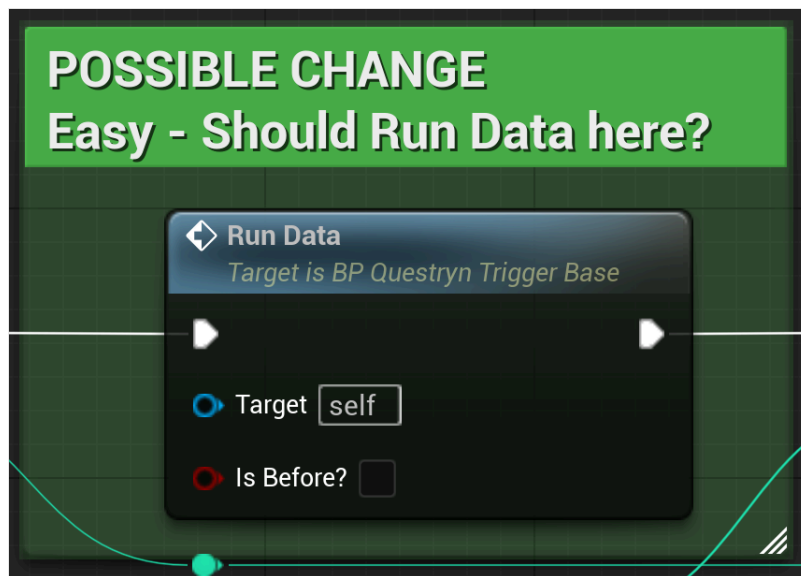
Throughout the example Blueprints you'll find color-coded comment boxes marking points meant for you to look at. **None of them are required for the plugin to work** — the example project runs as-is. They exist to tell you where and how to adapt it to your own game.

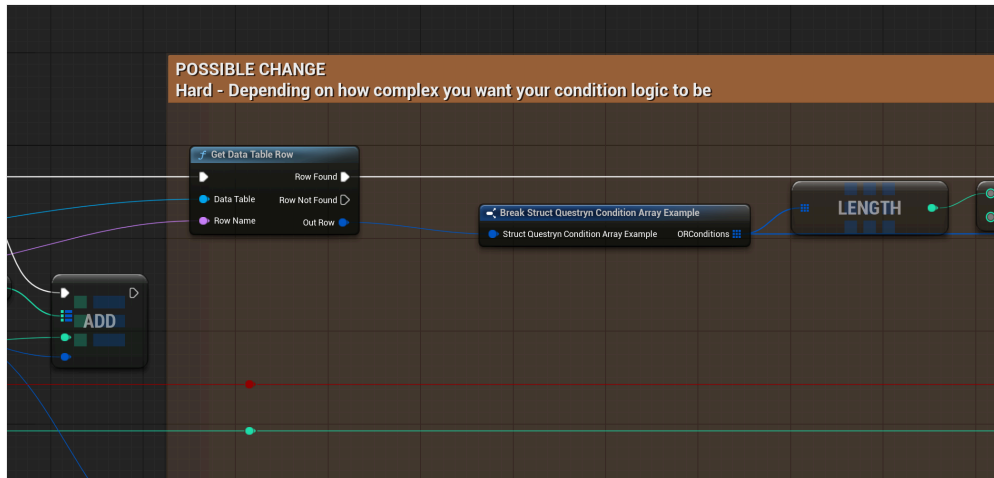
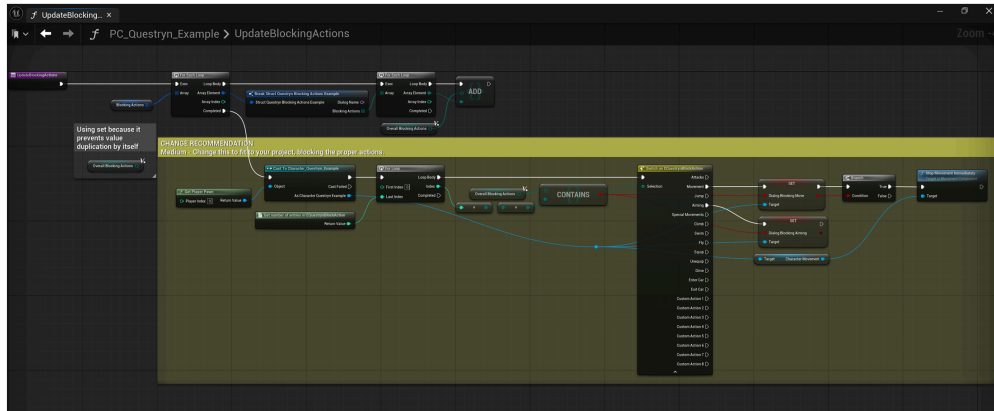
	Meaning
POSSIBLE CHANGE	Something you <i>can</i> change if it doesn't fit your project, or can remove entirely if you don't need the feature. Not a recommendation either way.
CHANGE RECOMMENDATION	Something the author suggests you adapt, specifically so you get the plugin's full potential in your own project (e.g. wiring conditions to your actual item/quest system instead of the example one).

Color marks **difficulty**, independent of the category above:

Color	Difficulty
● Green	Easy
● Yellow	Medium
● Orange	Hard — usually "hard" because it depends on how complex <i>you</i> want your own logic to be, not because the change itself is exotic.

Examples, straight from the Blueprints:



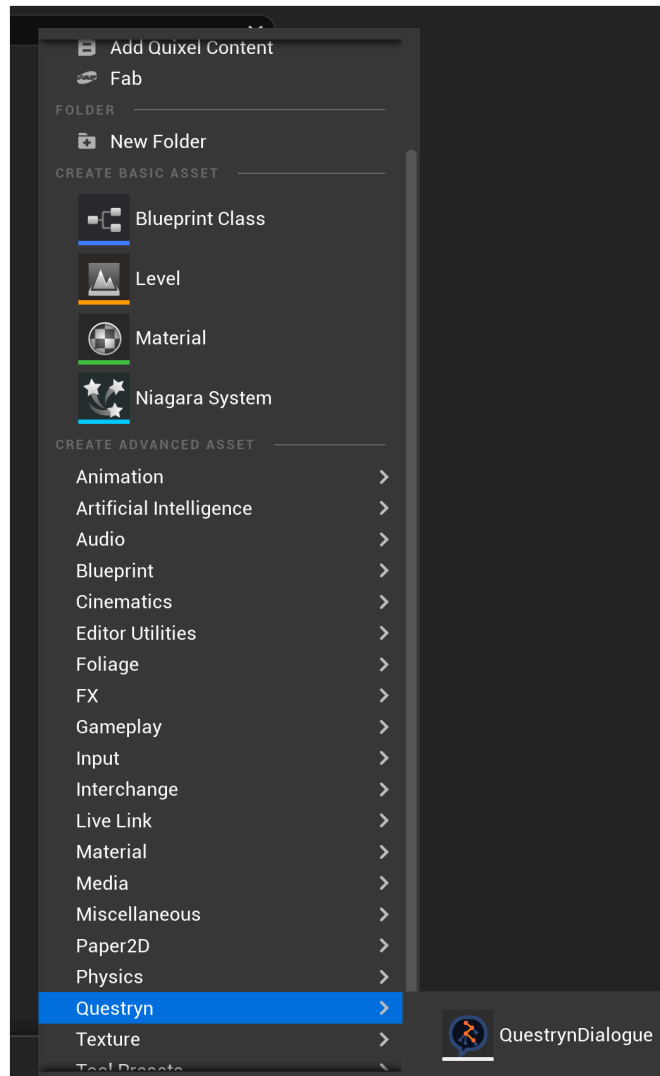


Keep this legend in mind for the rest of Section 5 — instead of repeating every single comment box, this guide summarizes what each function does and calls out the noteworthy ones.

4. The Editor Workflow

4.1 Creating a QuestrynDialogue Asset

A dialogue lives in its own asset type, `QuestrynDialogue`. Create one through the Content Browser's right-click menu — **Create Advanced Asset** → **Questryn** → **QuestrynDialogue** (or just type "Questryn" into the create-menu search):



This is the only supported way to create a dialogue. `QuestrynDialogue` does **not** appear as a selectable parent class under **Add** → **Blueprint Class** — that route would create an unrelated, empty Blueprint asset (opened in the generic Blueprint Editor, not the Questryn graph editor), not a usable dialogue.

4.2 Start Node

Every `QuestrynDialogue` asset has one fixed Start Node. Instead of hiding the dialogue's configuration in the Details panel, it's exposed directly on the node:

Get Sword

Dialogue Type:

Talk

Dialogue Tag:

getsword

Priority:

0

Lower Priority Behavior:

Stop

Input Blocks

None

Add Blocking Action

Movement

Context Tags

Add Context Tag

NPC

Sword

Description

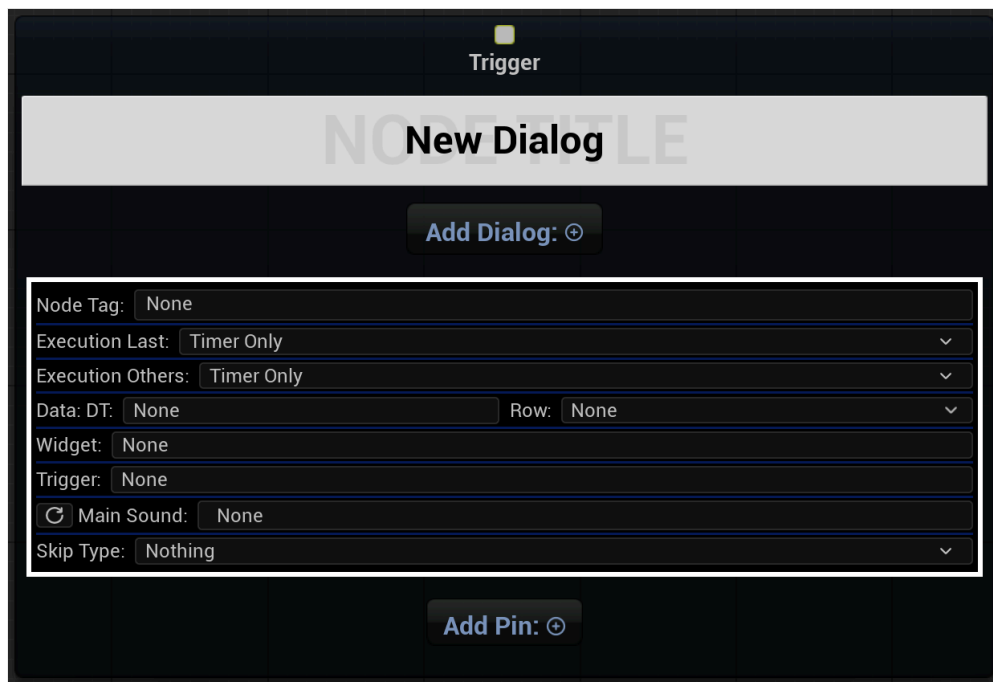
NPC asks you to fight the boss with his sword.

Start

- **Dialogue Type** — a category (Talk, Quest, Notification, etc.).
- **Dialogue Tag** — the identifier gameplay code uses to target this exact dialogue (`Start Dialogue` , `Force Pause/Resume/Destroy Dialogue` , `Execute Next Dialogue` — see 4.9 — all take a Dialogue Tag).
- **Priority + Lower Priority Behavior** — what happens to already-running dialogues when this one starts, if they're lower priority (e.g. Stop). This comparison is scoped to **Dialogue Type**: a higher-priority dialogue only forces the Lower Priority Behavior on running dialogues of the *same* Dialogue Type — a dialogue of a different type keeps running untouched, regardless of priority.
- **Input Blocks** — player actions blocked while this dialogue runs (Movement, Jump, Attacks, ...) — see how the example Player Controller/Character actually enforce these in 5.1–5.2.
- **Context Tags / Description** — free text used only by the Dialogue Browser (4.7); no runtime effect.

4.3 Dialog Node

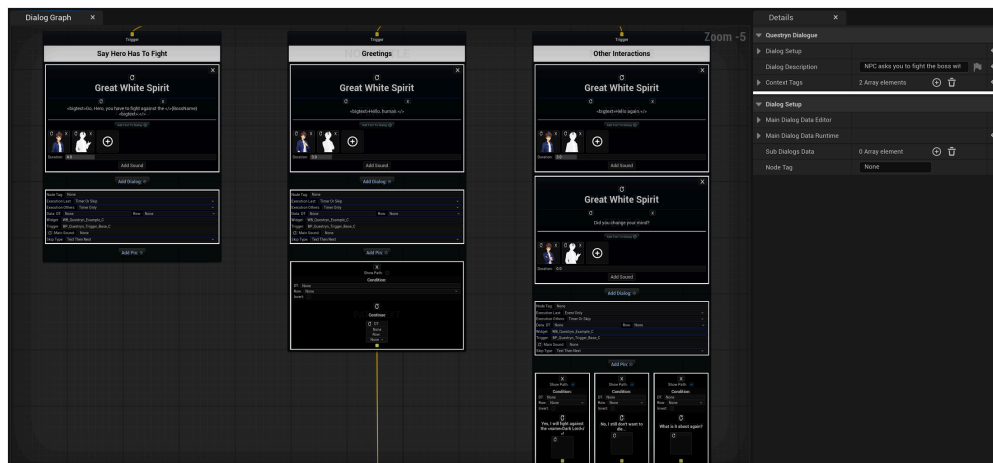
A Dialog Node is one "beat" of the conversation. Empty, it looks like this:



Each **SubNode** (one block per line spoken) has: dialog title (usually the speaker), text (raw rich-text markup, e.g. `<bigtext>Hello.</>`), one or more texture buttons (portraits/images, toggle-able between a direct value or a DataTable row), **Duration**, and **Add Sound**. "Add Text To Dialog" adds another line of text to the same SubNode; "Add Dialog" adds a whole new SubNode.

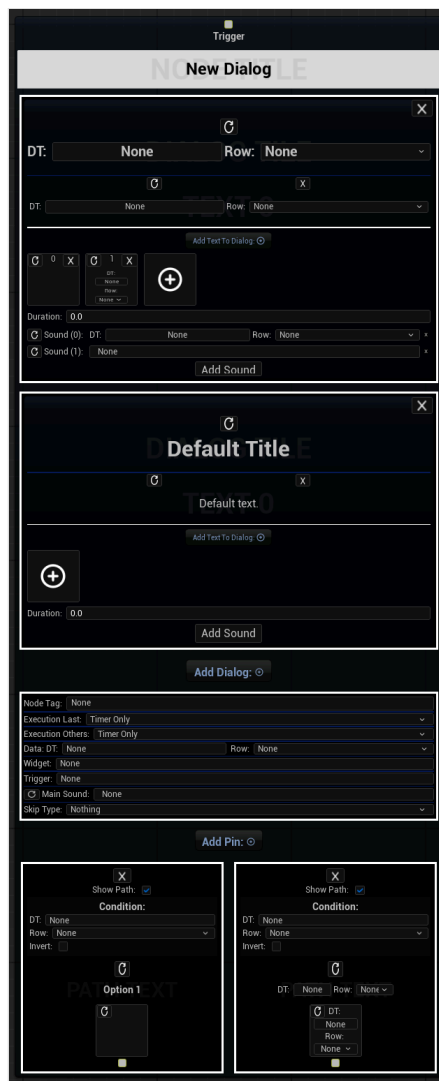
Below the SubNodes, the execution fields control pacing and behavior: **Node Tag** (an optional freeform name for this exact node — on its own it does nothing, but `Start Dialogue` (4.9) can target it to begin the dialogue right here instead of at the beginning, e.g. to resume a tutorial where the player left off), **Execution Last** / **Execution Others** (how the last vs. earlier lines advance — Timer, Timer or Skip, Skip, Event Only, Event with Time Limit), **Data DT/Row** (see `RunData` , 5.3), **Widget** (which widget class to spawn — defaults to the node's own if unset), **Trigger** (which Trigger class handles this node — see 5.3), **Main Sound**, and **Skip Type** (what a "skip" input does here: nothing, skip text, skip-then-advance, advance directly, jump to the last line, stop, or finish the dialogue).

A filled example, alongside the Details panel showing the same data:



4.4 Branching, Paths & Conditions

Every output pin of a Dialog Node is a **path**, and each path is its own customized pin widget — not the engine's default pin:

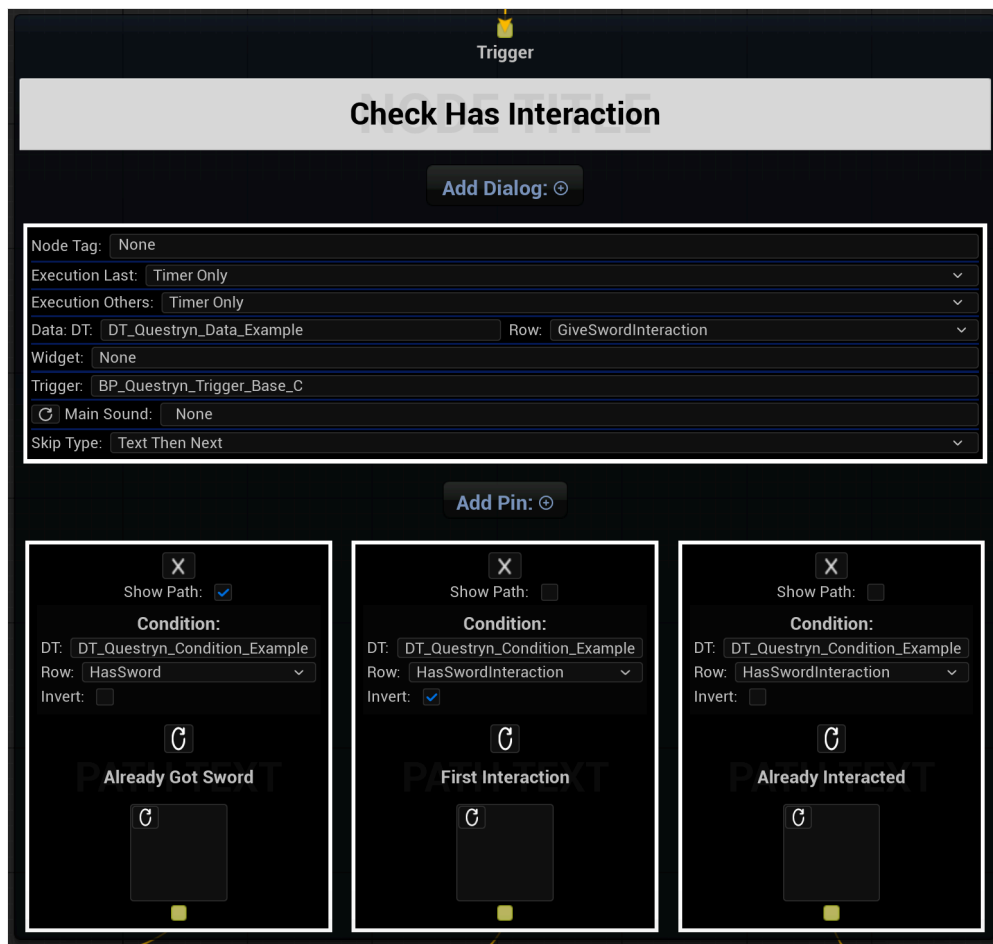


Graph connections follow one rule throughout: an output (a path) can only ever connect to a single input, but an input (a Dialog Node) can receive incoming connections from any number of outputs — so separate branches are free to converge back into a shared continuation node.

- **Show Path** — whether this path is presented to the player as a visible/selectable choice, or is an automatic/hidden path.
- **Condition** (DataTable + Row + **Invert**) — gates the path on a condition read from a DataTable row. See `CheckCondition` (5.3) for how the condition is actually evaluated.
- The path's own name (editable), and — when Show Path is checked — an icon representing the choice.

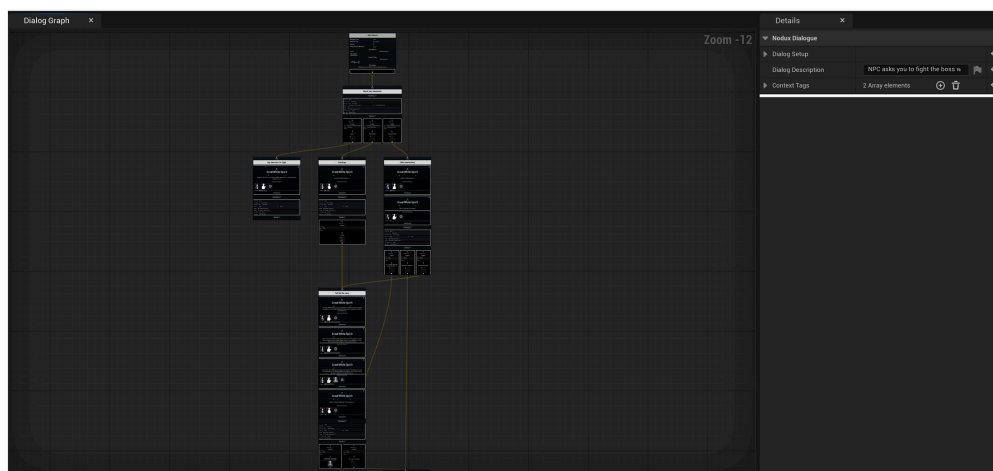
Almost any piece of content on a node (text, sound, texture, ...) can alternate between a **direct** value typed/selected right on the node, or a reference to a **DataTable row** — the small toggle icons switch between the two. This is what lets the same dialogue read from live game data instead of hardcoded text.

A Dialog Node doesn't need any SubNodes at all — it can be used purely as branching logic, e.g. to check several conditions and route to different follow-up nodes without showing anything to the player:



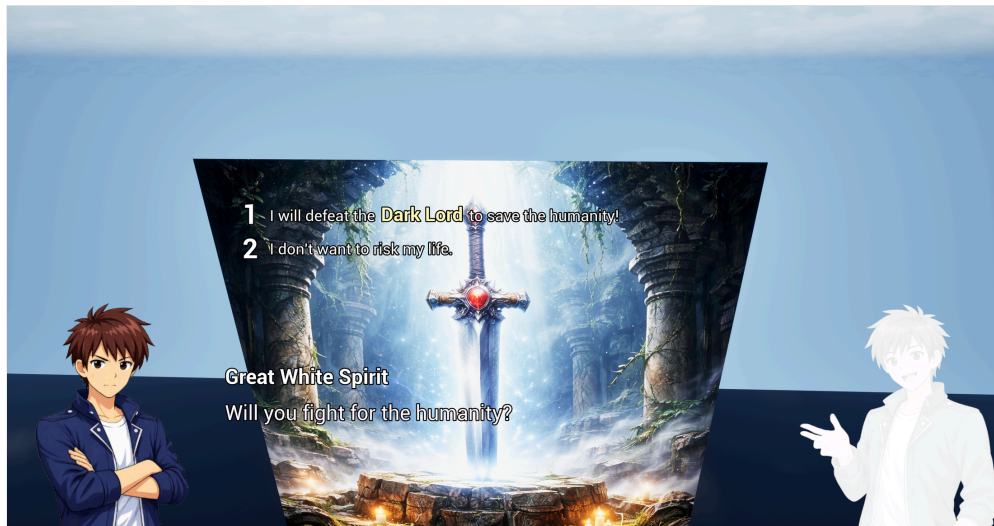
4.5 A Larger Dialogue Graph

Zoomed further out, here's part of a bigger dialogue in the same style — Start Node at the top, branching into a preparatory check (4.4) and from there into several further Dialog Nodes, with the graph continuing below what's shown here. Details panel on the right showing the asset-level setup:



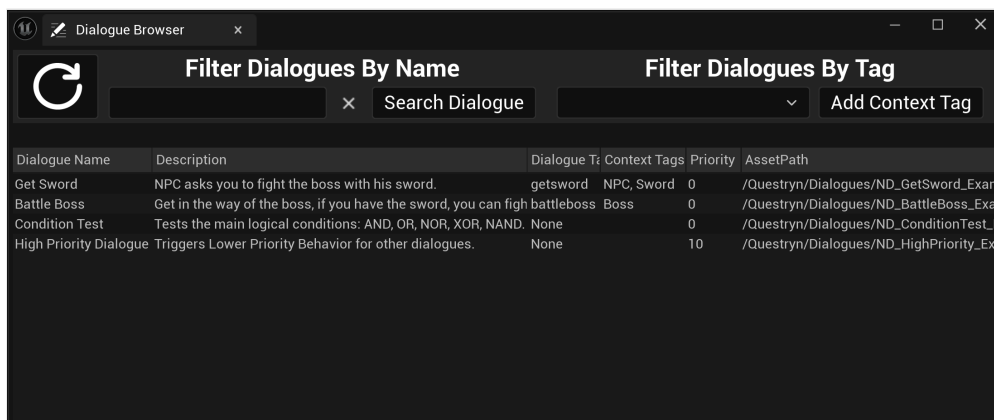
4.6 What the Player Sees

The same dialogue at runtime, using the example widget (portraits, title, numbered choices):



4.7 Dialogue Browser

As the number of `QuestrynDialogue` assets in a project grows, finding them by folder/filename in the Content Browser stops being practical. The **Dialogue Browser** (`Window` menu) lists every dialogue in the project and lets you filter by name or by Context Tag:



Columns: Dialogue Name, Description, Dialogue Tag, Context Tags, Priority, AssetPath.

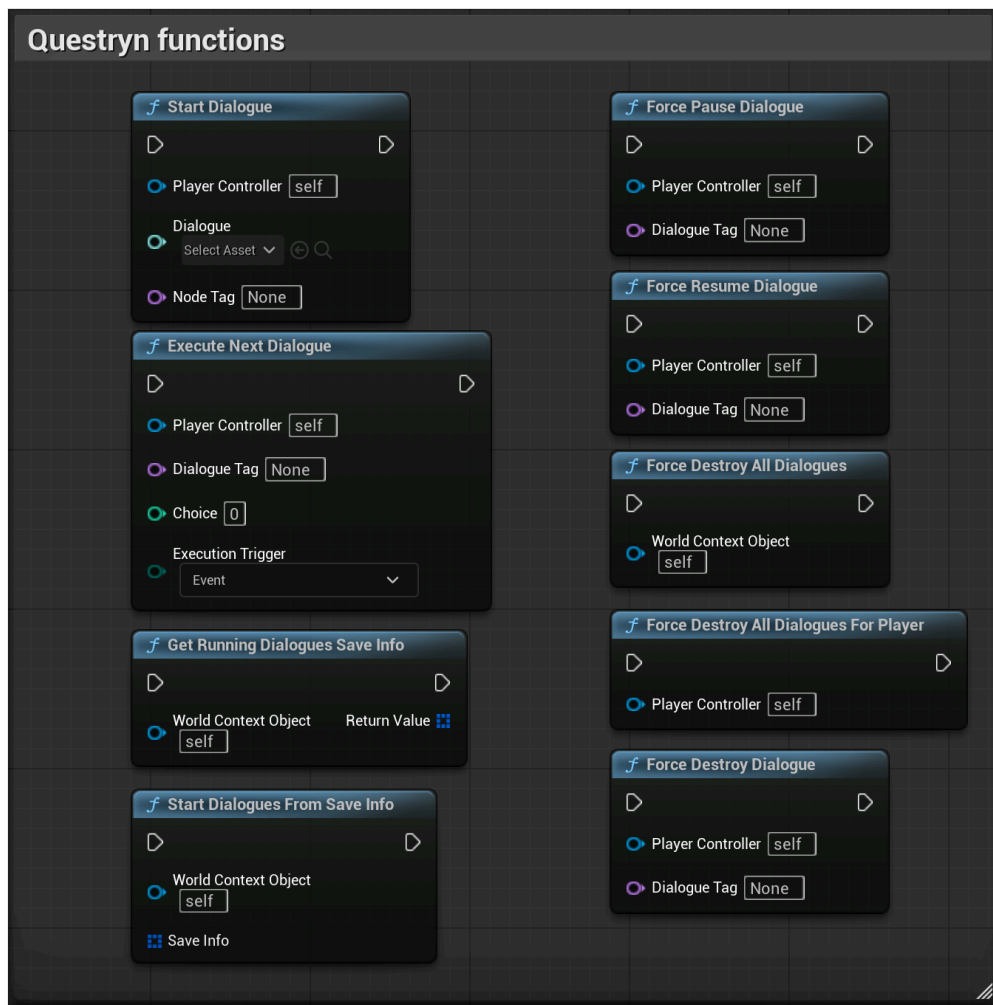
4.8 Find in Graph

The Dialogue Browser (4.7) finds a dialogue *asset* across the whole project; **Find** does the equivalent search *inside* the one dialogue graph you currently have open — useful once a graph grows past the point where scanning it visually for one specific node is practical.

It's a dockable tab (open by default alongside Details; reopen it anytime via **Ctrl+F** or the graph editor's `Window` menu). Type into the search box and results update as you type, matching against every Dialog Node's title, Node Tag, DATA table reference, Widget/Trigger class, and main sound, plus — per path option — its text/condition/texture, and — per line (SubDialogue) — its title, text(s), texture(s), and sound(s). Double-click a result, or press `Enter` with at least one result showing, to jump the graph view straight to that node.

4.9 Calling Dialogues From Anywhere

`UQuestrynFunctionLibrary` exposes Blueprint-callable functions to drive dialogues from any gameplay code — a Player Controller, a Trigger, a cutscene, etc.:



- **Start Dialogue** (Player Controller, Dialogue asset, Node Tag) — Node Tag is optional; leave it empty to start at the dialogue's normal beginning. If set, it searches the graph for a Dialog Node whose own **Node Tag** (4.3) matches and starts there instead; if no node has that tag, it falls back to the normal beginning.
- **Execute Next Dialogue** (Player Controller, Dialogue Tag, Choice, Execution Trigger)
- **Force Pause / Force Resume Dialogue** (Player Controller, Dialogue Tag)
- **Force Destroy Dialogue** (Player Controller, Dialogue Tag)
- **Force Destroy All Dialogues** (World Context) / **Force Destroy All Dialogues For Player** (Player Controller)
- **Get Running Dialogues Save Info** (World Context → array of `FQuestrynSaveData`, one entry per currently running dialogue across every local player: Dialogue asset, Dialogue Type, current Node Tag, and Local Player Index) — call this whenever your own save system writes a save, and persist the returned array in it.
- **Start Dialogues From Save Info** (World Context, array of `FQuestrynSaveData`) — the inverse of the function above: call it after your save system loads, passing back the array it got from **Get Running Dialogues Save Info**. Resolves each entry's Player Controller from its Local Player Index and calls **Start Dialogue** at the saved Node Tag; an entry whose Local Player Index no longer maps to a valid Player Controller (e.g. a split-screen player who isn't there anymore) is silently skipped.

Calling without a tag: any of these functions that takes a Dialogue Tag, when called with an empty/none tag, acts on the currently running dialogue with the highest priority — and if more than one shares that top priority, the most recently created one.

Save/load is your responsibility: Questryn deliberately has no save system of its own — the two functions above only get data in and out of memory. Where and how you persist the `FQuestrynSaveData` array (a savegame slot, a cloud save, whatever your project already uses) is entirely up to your own save system.

5. Blueprint Architecture of the Example Project

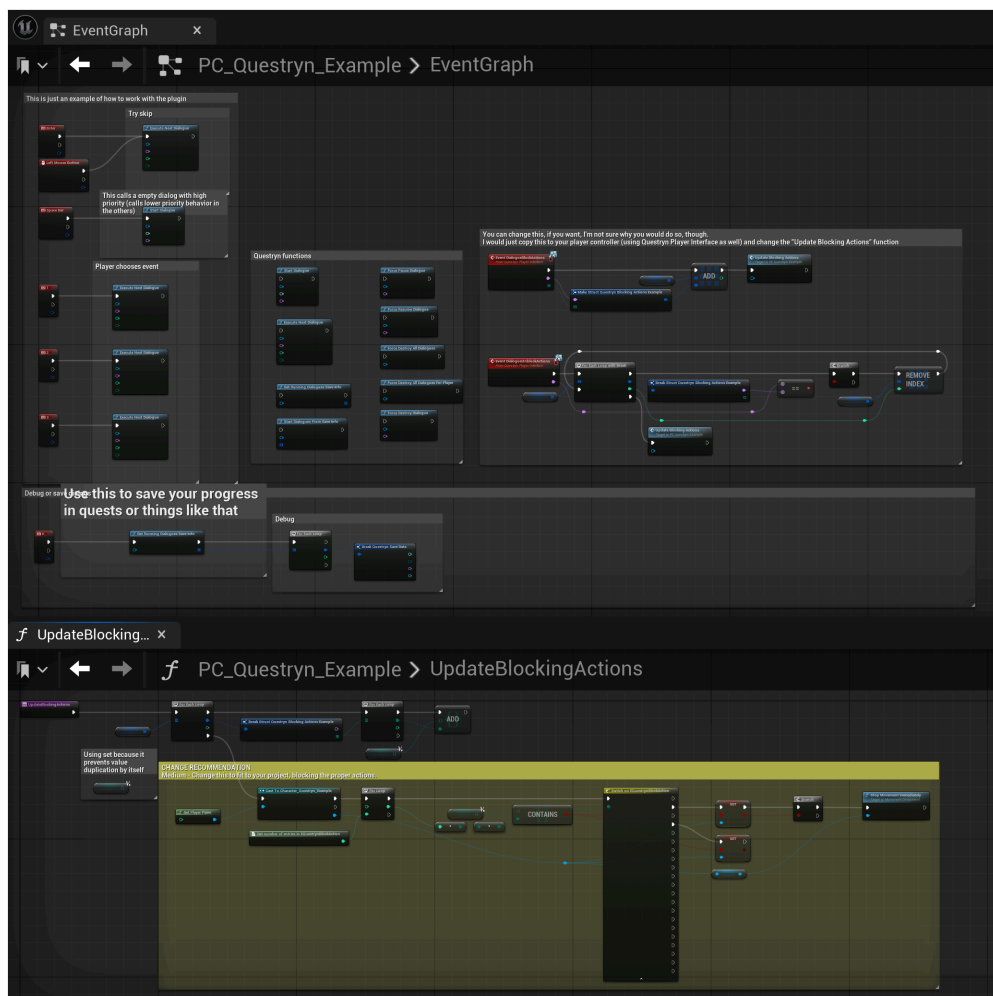
Everything in this section lives in **Content**, not in the plugin's C++ — it's Blueprint, yours to inspect, change, rename or delete (see Section 2). It's what turns a `QuestrynDialogue` asset into an actual on-screen, interactive conversation. Comment-box call-outs below use the legend from Section 3.

5.1 PC_Questryn_Example (Player Controller)

UpdateBlockingActions — takes the union of `Blocking Actions` currently requested by every running dialogue (`EQuestrynBlockAction`: Attacks, Movement, Jump, Special Movements, Climb, Swim, Fly, Equip, Unequip, Drive, Enter Car, Exit Car, Custom Action 1–8) into a `Set` (chosen specifically to avoid duplicate entries), then hands the result to the Character (5.2) to actually enforce.

● **CHANGE RECOMMENDATION** — *Medium*: adapt this so it blocks the actions that actually matter for your project.

Event Graph — mostly example/demo wiring, meant to be copied and adapted rather than used verbatim. Shown below together with `UpdateBlockingActions` itself:

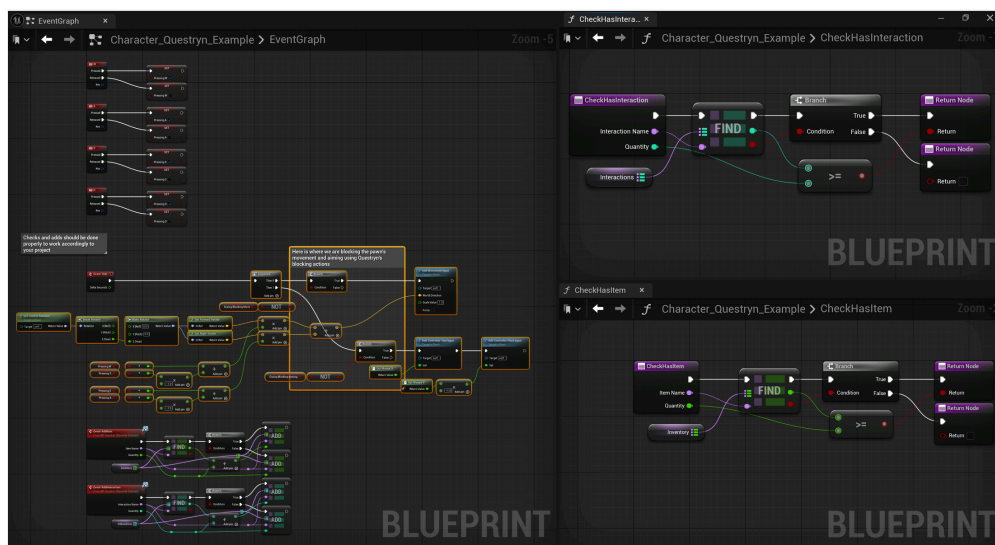


- Left-click → **Try Skip** on the current dialogue's *Execute Next Dialogue*.

- Space → starts an empty, high-priority dialogue (demonstrates the priority/"Lower Priority Behavior" system pushing other dialogues aside).
- Keys 1/2/3 → **Execute Next Dialogue** with Choice 0/1/2 (answering a path-choice prompt).
- `Event DialogBlockActions` / `Event DialogUnblockActions` (from the Qwestryn Player Interface) — maintain a per-dialogue array of requested blocking actions and call `UpdateBlockingActions` whenever it changes.
- **K** → debug: dumps the data of every currently running dialogue via `Get Running Dialogues Save Info` to the log.
- A "Qwestryn functions" region shows every `UQwestrynFunctionLibrary` call available (same list as 4.9), ready to wire up.

5.2 Character_Qwestryn_Example (Pawn)

Event Graph — tracks WASD as booleans and, on Tick, applies movement/looking input built from those booleans — standard third/first-person movement, included only so the blocking demo has something to block. The part relevant to Qwestryn: movement input and mouse look are each gated behind `NOT DialogBlockingMove` / `NOT DialogBlockingAiming`, two booleans driven by `PC_Qwestryn_Example`'s blocking-actions handling.



- "Checks and adds should be done properly to work accordingly to your project" — the movement/aiming block here is a minimal example; a real project usually needs more granular checks.

Functions — small inventory/interaction helpers, exposed through the `BPI_Qwestryn_Character_Example` interface (5.5) so `FilterPathOptions` / `CheckCondition` (5.3) never need to know the concrete class implementing them, visible on the right of the same screenshot above:

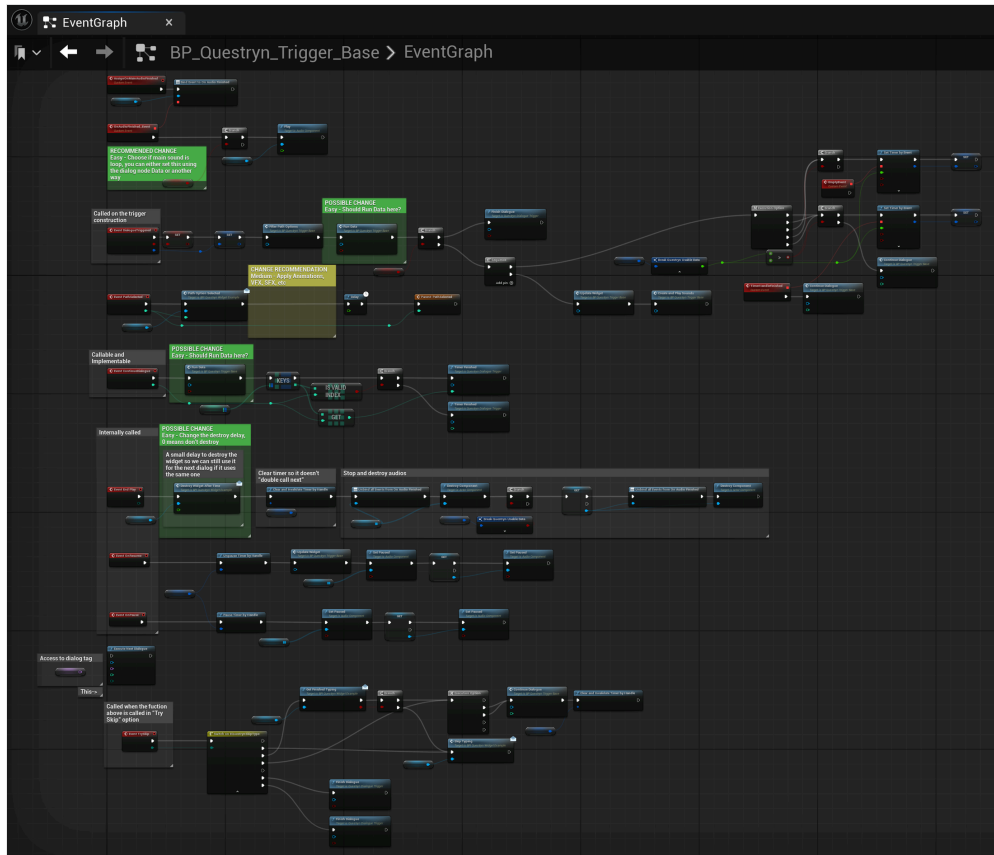
- `Inventory` (Map: item name → quantity) and `Interactions` (Map: interaction name → done/not done) are the two backing variables.
- `CheckHasItem` / `CheckHasInteraction` (implemented as Functions) and `AddItem` / `AddInteraction` (implemented as Events) — straightforward find/add on those maps, wired up as the interface's four functions.

This is the class most likely to be swapped for your own character early on — since `BP_Qwestryn_Trigger_Base` only ever reaches it through the interface, any actor implementing `BPI_Qwestryn_Character_Example` can stand in for it without touching the Trigger Blueprint at all.

5.3 BP_Questryn_Trigger_Base — the Main Extension Point

This is **the** class to look at first. Every Dialog Node references a Trigger class (4.3); this is the example implementation of `QuestrynDialogueTrigger`, and the one place where dialogue data, conditions, text/texture/sound resolution, widget updates and gameplay side-effects all meet. When you build your own, copy this Blueprint rather than subclassing it — copies are the supported customization path here.

Event Graph — the core lifecycle, plus pause/resume, skip, and tag-based access, all in the same graph:

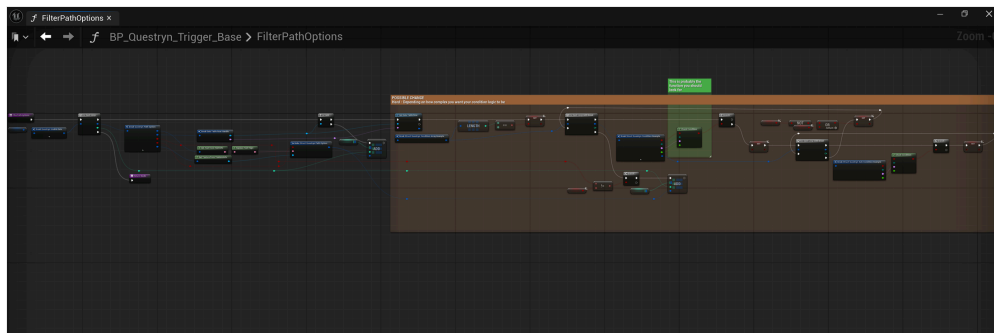


- `AssignOnMainAudioFinished` binds the main audio component's finish event to `OnAudioFinished_Event`, which — ● *Easy*, **CHANGE RECOMMENDATION** — replays the main sound if `Loop Main Sound` is set (you can drive this from the Dialog Node's data instead).
- Event `DialogTriggered` (called on trigger construction) — calls `FilterPathOptions` (below), then `RunData` (● *Easy*, POSSIBLE CHANGE: "should Run Data run here?"). If the node is Trigger-Only, it finishes the dialogue right away (with Trigger Events). Otherwise it calls `UpdateWidget` + `CreateAndPlaySounds`, and — depending on the node's Execution Option — sets a timer to advance automatically.
- Event `PathSelected` (a protected native event — implementable in Blueprint, but only callable from within the class hierarchy, not from arbitrary external Blueprints) — calls `PathOptionSelected` on the dialogue widget, waits (● *Medium*, CHANGE RECOMMENDATION: "Apply Animations, VFX, SFX, etc." during this delay), then calls the parent's `PathSelected` to continue.
- Event `ContinueDialog` (same protected-native-event pattern) — `RunData` again (● *Easy*, same "should Run Data run here?" note), then finishes the current line by checking the `Choice` is valid and calling `TimerFinished`.
- Event `End Play` — destroys the widget after a short delay (● *Easy*, POSSIBLE CHANGE: "change the destroy delay" — kept so the same widget instance can still be reused by the *next* dialogue if it uses one), clears the timer, and unbinds/destroys the audio components (only destroying the shared main component if this is the last sub-dialogue).

- `Event OnPause / Event OnResume` — pause/unpause the internal timer and every audio component (main + secondary); `OnResume` also refreshes the widget.
- `Event TrySkip` — switches on the node's `Skip Type` (`EQuestrynSkipType`: Nothing, Text Only, Text Then Next, Next Directly, Skip to Last Sub Dialog, Stop Dialog, Finish Dialog) and reacts accordingly — typically checking whether typing has finished (`Get Finished Typing`) before deciding whether to skip the typing animation or advance.
- Shows `Execute Next Dialogue` being called by Dialogue Tag as a plain access-pattern example.

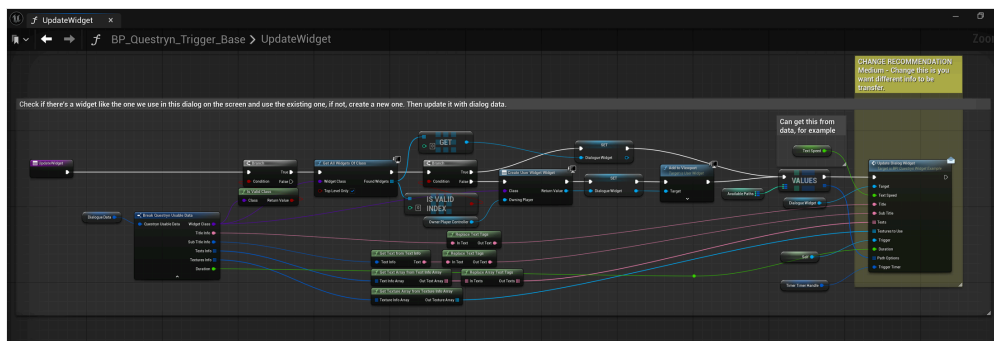
Functions

`FilterPathOptions` — evaluates every path's condition(s) and returns only the paths currently valid, reaching the controlled pawn's condition checks through `BPI_Questryn_Character_Example` (5.5) rather than any hard-coded character class:



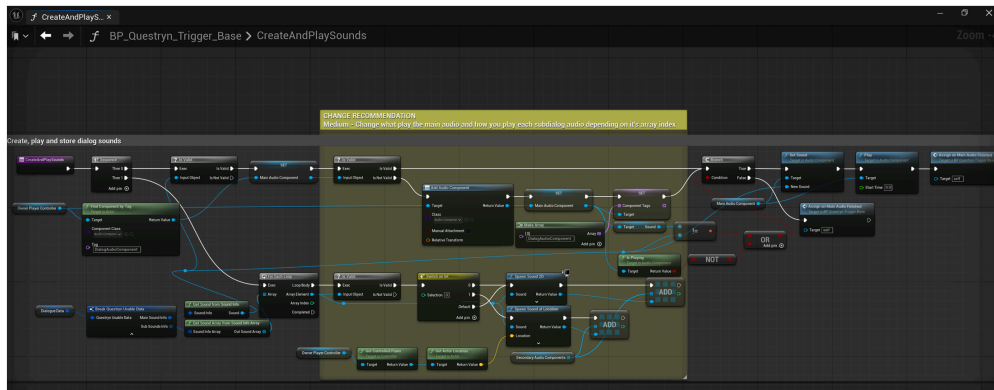
- **POSSIBLE CHANGE** (Hard, "depending on how complex you want your condition logic to be") — the actual filter logic: conditions are grouped as **an array of OR groups, each containing an array of AND conditions** — a path is valid if *any* OR group has *all* of its AND conditions true. This is the function to look at if you need more elaborate condition logic than AND-of-ORs.

`UpdateWidget` — finds an existing dialogue widget on screen and reuses it, or creates a new one; gathers the node's title/subtitle/text(s)/texture(s) (resolving `{Tag}` substitutions and DataTable references along the way) and hands it all to the widget through the `BPI_Questryn_Widget_Example` interface:



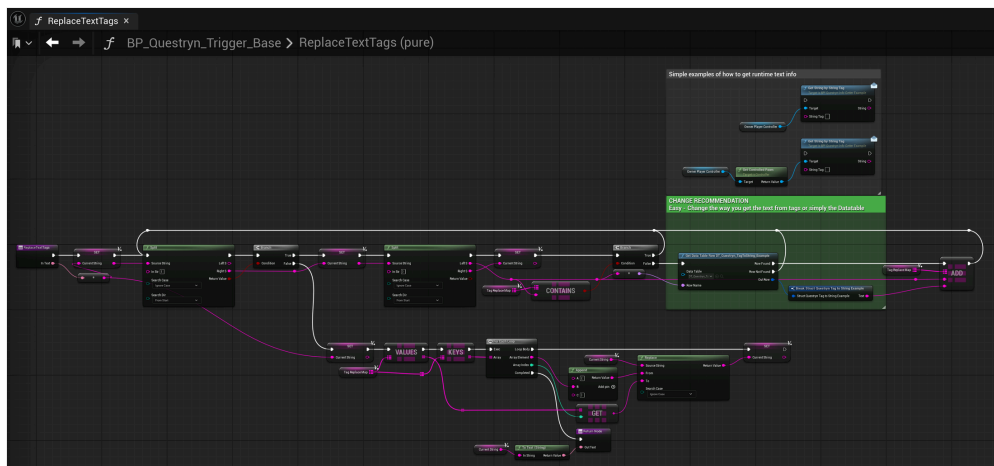
- **CHANGE RECOMMENDATION** (Medium) — adapt `UpdateDialogWidget` (the interface call) if you want different information transferred to your own widget.

`CreateAndPlaySounds` — creates and plays the main sound plus each sub-dialogue's sound as dynamically added Audio Components, choosing 2D vs. 3D-at-location by array index in the example:



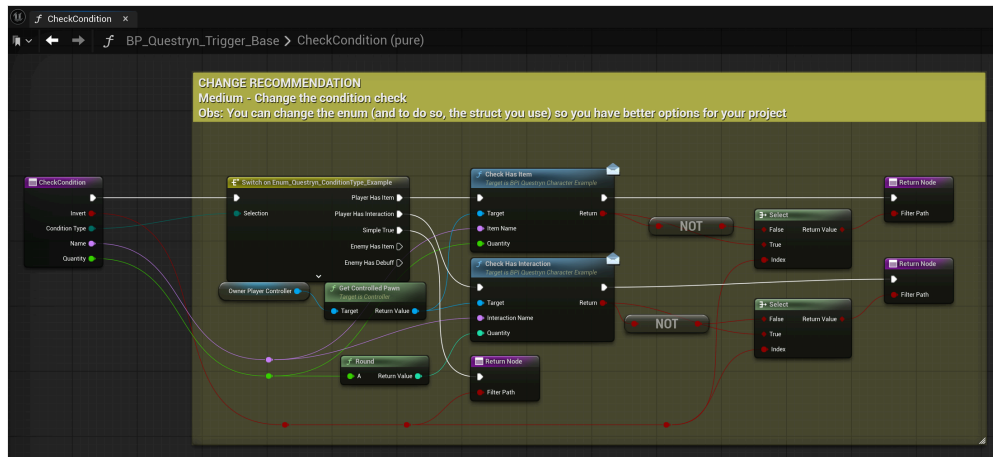
- **CHANGE RECOMMENDATION** (Medium) — change what plays the main audio, and how each sub-dialogue's sound is played depending on its array index.

`ReplaceTextTags` (pure) — finds `{Tag}`-style placeholders in a string (distinct from `<RichText>` markup — see the plugin's design notes) and replaces each with a value looked up from `DT_Questryn_TagToString_Example` :



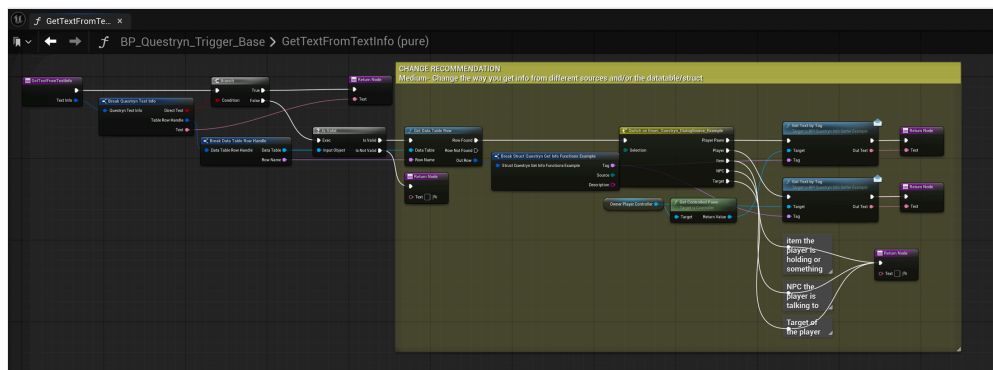
- **CHANGE RECOMMENDATION** (Easy) — change where the replacement text comes from (a different table, a function, whatever fits your project) instead of this one DataTable.

`CheckCondition` (pure) — switches on `Enum_Questryn_ConditionType_Example` (**Has Item, Has Interaction, Simple True**), calls `CheckHasItem` / `CheckHasInteraction` on the controlled pawn through `BPI_Questryn_Character_Example` (5.2, 5.5) rather than casting to a specific class, and applies **Invert** if set:



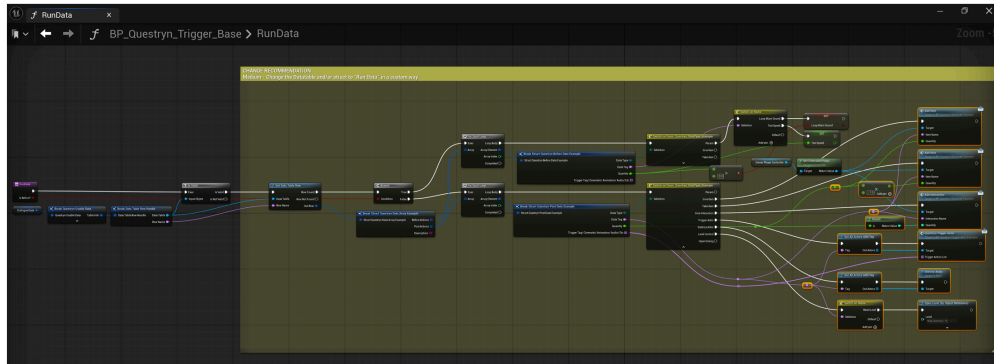
● **CHANGE RECOMMENDATION** (Medium) — change the condition check itself; you can also extend the enum (and the struct backing it) for more condition types.

`GetTextFromTextInfo` / `GetTextureFromTextureInfo` / `GetSoundFromSoundInfo` (pure) — the direct-vs-DataTable resolvers behind every "soft" info field (4.4). If the field is set to a direct value, it's returned as-is; otherwise a DataTable row supplies a **Source** + **Tag**, and an `Enum_Questryn_DialogSource_Example` switch routes the lookup to the right object (the player pawn, the player controller, an item reference, the NPC, or a generic target) via `BPI_Questryn_InfoGetter_Example` :



● **CHANGE RECOMMENDATION** (Medium, all three functions) — change how you fetch info from different sources and/or the DataTable/struct shape.

`RunData` — the general-purpose gameplay side-effect dispatcher, driven by a DataTable row referenced from the node's **Data: DT/Row** field (its **Give Item** / **Give Interaction** actions call `AddItem` / `AddInteraction` through `BPI_Questryn_Character_Example`, same as `CheckCondition`). Each row holds two ordered lists — **Before Actions** and **Post Actions** — and `RunData`'s **Is Before?** input picks which list runs (once when the node triggers, once when the player finishes/continues past it). Each action has a **Data Type** (`Enum_Questryn_DataType_Example`: **Give Item**, **Take Item**, **Give Interaction**, **Trigger Actor**, **Destroy Actor**, **Level Control**, **Open Dialog**), a **Data Tag** identifying what it applies to, a **Quantity**, and an extra field for a trigger tag/cinematic/animation/audio reference depending on the type:



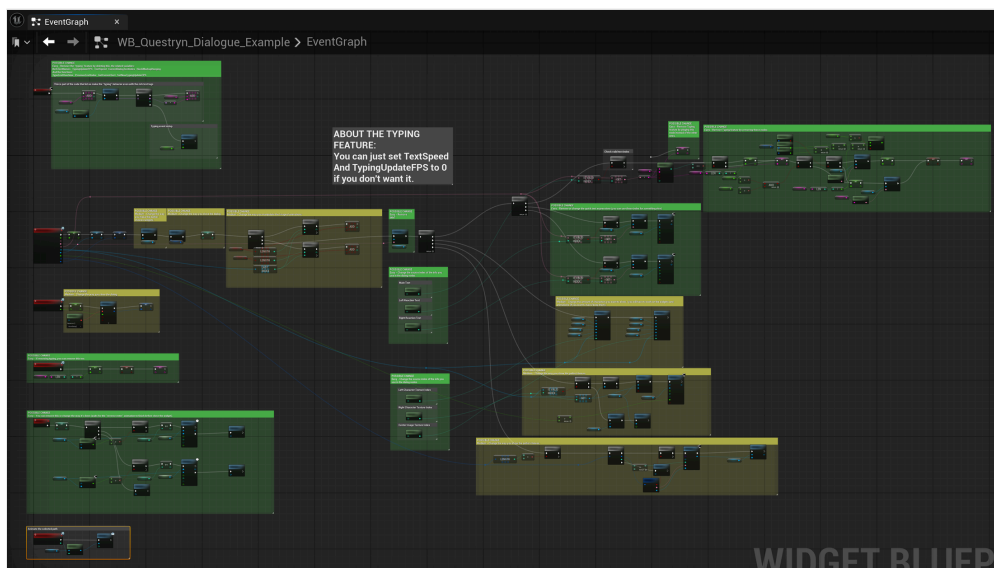
● **CHANGE RECOMMENDATION** (Medium) — adapt the DataTable/struct and the actions themselves to your project; there's a couple of special-cased data tags in the example too (e.g. one that toggles main-sound looping, one that resets the level) — check the graph directly if you want to see exactly how those are wired.

This function alone can drive almost anything — setting a parameter, giving/removing an item, spawning/destroying/otherwise manipulating an actor, running level commands, opening another dialogue, calling a specific function some other way, etc. — the example only covers a handful of simple cases.

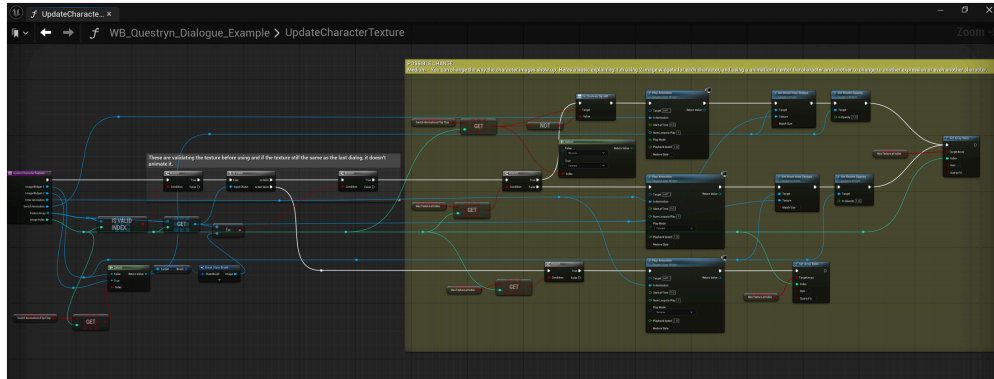
5.4 WB_Questryn_Dialogue_Example — the Dialogue Widget

This is the widget doing the actual work of turning a Dialog Node's data into a fully functional, animated dialogue screen: text/array/texture processing, the typing-machine effect, positioning, choice widgets. As of this writing it has **16 POSSIBLE CHANGE points in the Event Graph and 1 in UpdateCharacterTexture** — all related to presentation/behavior choices, none required.

The typing effect is implemented as its own self-contained block — variables `RichTextNames`, `TypingUpdateFPS`, `TextSpeed`, `CurrentDialogTextIndex`, `HasAMarkupHanging`, and functions `TypeTextOvertime`, `PreviousTextIndex`, `GetCurrentText`, `SetNewTypingUpdateFPS` — and can be removed entirely if you don't want it (● *Easy*, POSSIBLE CHANGE, called out wherever the typing-driven text node is used instead of a plain one). Other notable customization points in the same Event Graph: how the dialogue closes (waits for a "reverse enter" animation before destroying the widget — ● *Easy* to change or remove), how paths/choices are shown, how character portrait images and their enter/expression animations are swapped, the "quick text expression" shortcuts, and — twice — a set of `Make Literal Int` constants (`Main Text` / `Left Reaction Text` / `Right Reaction Text`, and `Left` / `Right` / `Center Character Texture Index`) that simply say *which array index* the widget reads for each purpose, in case you rearrange the source arrays:



`UpdateCharacterTexture` — validates whether a portrait texture actually changed since the last line before playing any animation for it (so it doesn't needlessly replay), then updates the image and plays an "enter" or "switch" animation. The example uses two Image widgets per character (one to animate the incoming image over the outgoing one):



- **POSSIBLE CHANGE** (Medium) — you can change how character images show up entirely; this is just one way to do it (2 image widgets, animate-in on enter, animate-change on expression change).

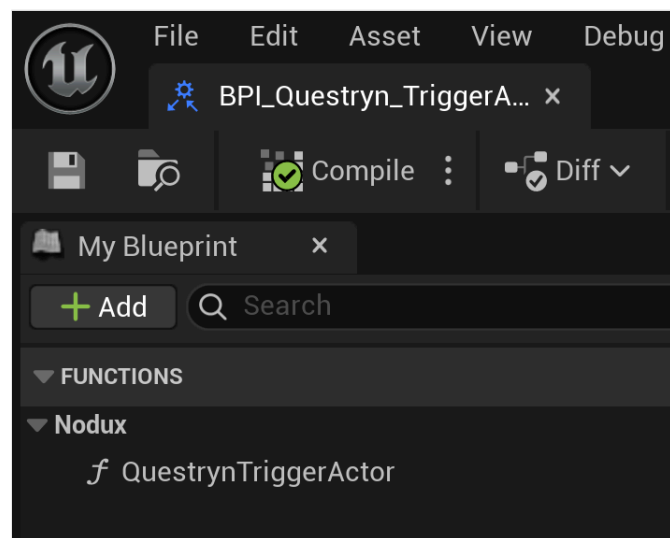
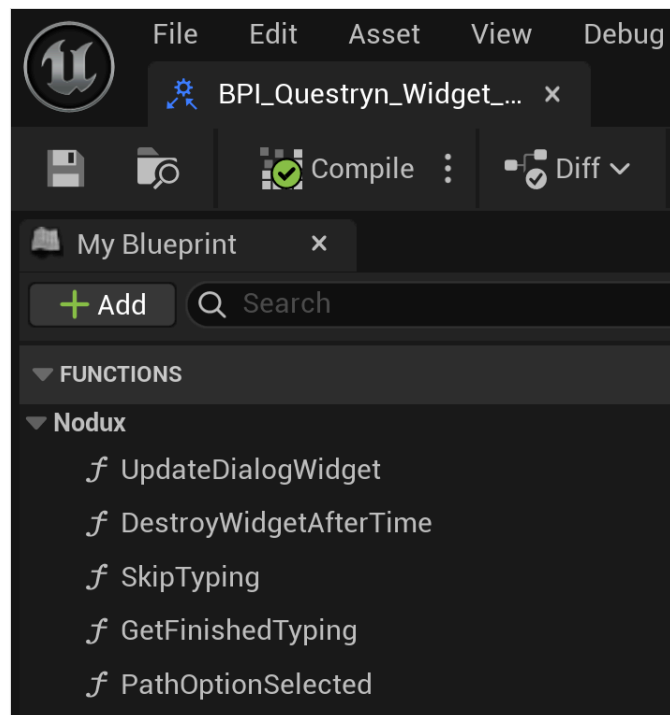
Companion widgets, each with a narrow job:

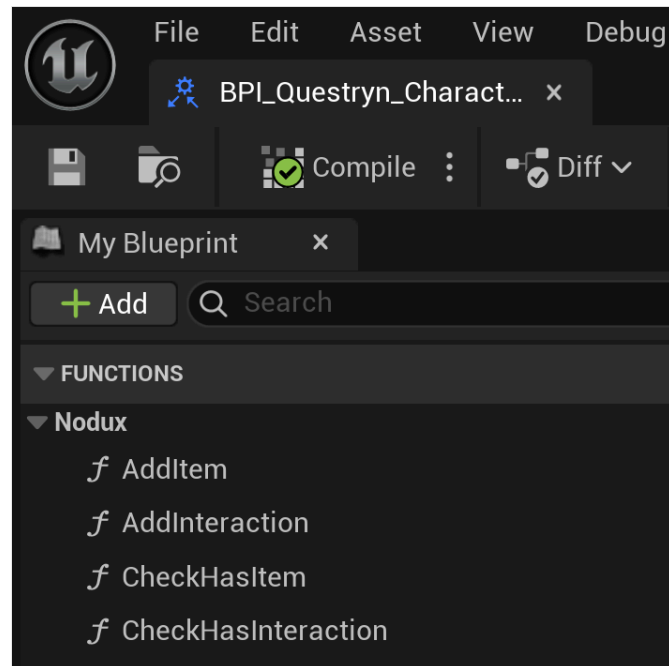
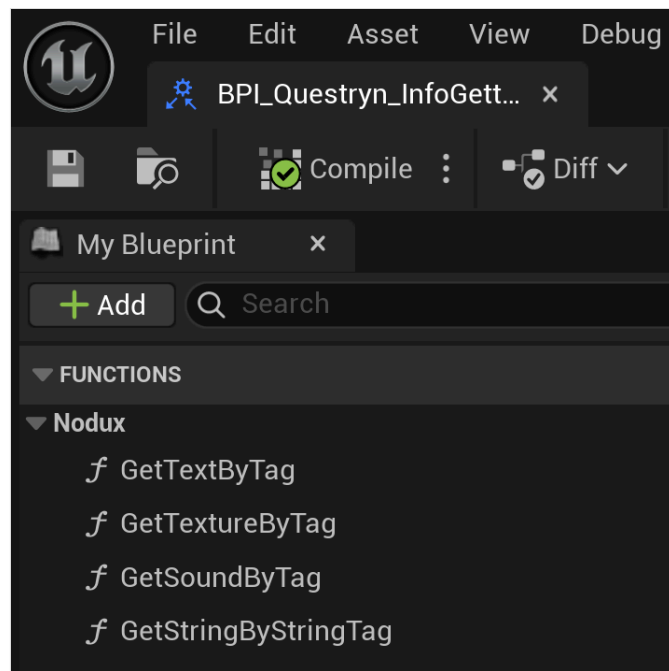
- `WB_Questryn_Notification_Example` — shows a single notification (image + text).
- `WB_Questryn_NotificationWindow_Example` — arranges multiple Notifications on screen.
- `WB_Questryn_PathOption_Example` — a single path/choice entry; stays on screen (parented inside `WB_Questryn_Dialogue_Example`) until the player picks one, then plays a short selection animation (triggered from `WB_Questryn_Dialogue_Example`, see `Event PathSelected` in 5.3).

5.5 The Four Interfaces

These exist specifically so `BP_Questryn_Trigger_Base` doesn't need to know the concrete class of the widget, the trigger actor, the character, or wherever info comes from — swap in your own classes that implement the same interface and the Trigger keeps working unchanged.

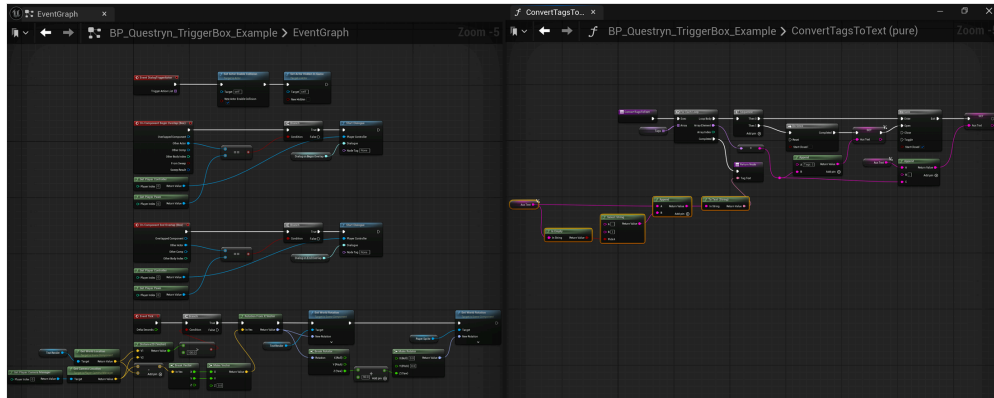
Interface	Functions	Implemented by (example)	Called from
<code>BPI_Questryn_Widget_Example</code>	<code>UpdateDialogWidget</code> , <code>DestroyWidgetAfterTime</code> , <code>SkipTyping</code> , <code>GetFinishedTyping</code> , <code>PathOptionSelected</code>	<code>WB_Questryn_Dialogue_Example</code>	<code>BP_Questryn_Trigger_Base</code> (<code>UpdateWidget</code> , <code>TrySkip</code> , <code>Event End Play</code> , <code>Event PathSelected</code>)
<code>BPI_Questryn_TriggerActor_Example</code>	<code>QuestrynTriggerActor</code>	Any actor you want <code>RunData</code> 's Trigger Actor / Destroy Actor action to reach	<code>BP_Questryn_Trigger_Base</code> (<code>RunData</code>)
<code>BPI_Questryn_InfoGetter_Example</code>	<code>GetTextByTagAndIndex</code> , <code>GetTextureByTagAndIndex</code> , <code>GetSoundByTagAndIndex</code>	Player Pawn / Player Controller / item / NPC / target, as needed	<code>BP_Questryn_Trigger_Base</code> (<code>GetTextFromTextInfo</code> and its Texture/Sound siblings)
<code>BPI_Questryn_Character_Example</code>	<code>CheckHasItem</code> , <code>CheckHasInteraction</code> , <code>AddItem</code> , <code>AddInteraction</code>	<code>Character_Questryn_Example</code> (5.2)	<code>BP_Questryn_Trigger_Base</code> (<code>FilterPathOptions</code> , <code>CheckCondition</code> , <code>RunData</code>)





5.6 BP_Questryn_TriggerBox_Example — Putting It All Together

A concrete, minimal example of triggering a dialogue from the world: an overlap volume that starts the dialogue on `Begin Overlap` and ends it on `End Overlap` (each guarded by a check that the overlapping actor is actually the player pawn), plus an unrelated bit of polish — a world-space prompt text that continuously faces the player camera (billboard/look-at, recalculated on Tick) — and its own small pure helper, `ConvertTagsToText`, which turns an array of gameplay tags into a single display string:



This is a good template to copy when building your own dialogue-starting actors (NPCs, interactables, cutscene triggers, ...).

6. Where to Start

If you only read one section besides this one, make it **5.3 (BP_Questryn_Trigger_Base)** — it's where dialogue data, conditions, and every other system in this guide actually connect. A practical order:

1. Follow Section 2 and get your own copies of the classes you need in place.
2. Implement `BPI_Questryn_Character_Example` (5.5) on your own character/condition system, so `FilterPathOptions` and `CheckCondition` reach it.
3. Point `GetTextFromTextInfo / Texture / Sound` and the `BPI_Questryn_InfoGetter_Example` implementations at your own data sources.
4. Adapt `RunData` to the gameplay actions your project actually needs.
5. Only then, if you want a different look and feel, work through `WB_Questryn_Dialogue_Example` 's presentation-level `POSSIBLE CHANGE` points.

Everything else in Sections 4–5 keeps working as shipped in the meantime.

7. LLM-Assisted Dialogue Authoring

Questryn ships a second file alongside this one — `Questryn_LLM_Authoring_Prompt.md` — that you're not meant to read yourself. Instead, copy the whole thing into a chat with an AI assistant (ChatGPT, Claude, etc.) together with the story or script you want turned into a dialogue. The AI reads it and then asks *you* plain-language questions about your setup and your story — you never need to open the file to know what's going on. Behind the scenes, it teaches the AI Questryn's data model and how to turn your answers into a dialogue graph: a human-readable plan for you to build by hand, or a ready-to-run Python generation script if the AI has Editor Python execution access to your project.

7.1 Getting the prompt file

Both documents — plus the community Discord — are reachable from the Editor without hunting through folders — **Help menu** → **Questryn**:

- **Questryn User Guide** — opens this document (PDF).
- **Copy Questryn LLM Prompt to Clipboard** — copies the whole file's text, ready to paste straight into an AI chat. There's no separate "open" option — you're not meant to read the file yourself, just hand it to an AI, so copying straight to clipboard is the only path.
- **Join the JN Softworks Discord** — opens the support server (see Section 8).

7.2 The Setup Table and Profile Table (optional, but recommended)

Two kinds of DataTable make the AI's job — and yours — easier, by letting you describe your reusable content once, ahead of time, instead of answering questions about it table-by-table in chat:

- **Setup Table** — one row per reusable DataTable you want the AI to draw on: conditions that gate a branch, sound/text/image lookups, generic gameplay data, or a Profile Table (below). Each row just says which category the table belongs to, a reference to the table itself, an optional description of what it's for, and — if you only want specific rows available — their names. Create one via **Content Browser** → **Add** → **Miscellaneous** → **Data Table**, Row Type `QuestrynLLMSetupEntry`. A filled-in example ships at `/Questryn/Data/DT_Questryn_LLMSourceTable_Example`.
- **Profile Table** — one row per reusable "node behavior" preset: which pacing/skip settings, Trigger class, and Widget class a node should use (the same fields described in 4.3), bundled under a name like `Default` or `Notification` so you can just say "use the Notification profile" instead of setting all four fields by hand on every node. Row struct `FQuestrynNodeProfile`, example table `/Questryn/Data/DT_Questryn_NodeProfile_Example`. This is purely an authoring-time convenience for the AI's generation script — it's not something the graph editor itself has a "Profile" picker for, so you can also ignore it entirely and set each node's fields by hand as usual.

Hand the AI your Setup Table directly if it has Editor Python access to your project. Otherwise, run a small Python script bundled inside `Questryn_LLM_Authoring_Prompt.md` once — Unreal Editor → **Tools** → **Execute Python Script...** — which writes a JSON file into your project's `Saved` folder; open that file and paste its contents into the chat instead. Either way, you fill in the setup once, in the DataTable editor, rather than one table at a time in chat.

Note that the **Trigger** field on both Dialog Nodes and Profile rows only ever offers actual Trigger classes (e.g. your copy of `BP_Questryn_Trigger_Base`) — the plugin's own `QuestrynDialogueTrigger` base class is abstract and deliberately never shows up as a selectable option.

8. Sharing Your Game (Optional)

Neither of these affects your license in any way (see `LICENSE.txt`, Section 4) — they're just two ways to let us know Questryn is part of your game, if you'd like to:

- **Drop the "Made with Questryn" badge into your credits.** A ready-to-use, transparent PNG ships at `Docs/Badge/QuestrynBadge.png` (also copied into `Badge/` by the **Add Starter Content** button, Section 2). It's designed for dark backgrounds.



- **Post in #showcase on the JN Softworks Discord** (Help menu → Questryn → Join the JN Softworks Discord, or the link in `LICENSE.txt`) when your game ships. We regularly feature community projects there and on JN Softworks' own channels.

